

Circuits Read off the Weights

What a transformer computes, before you run it on anything

Where we left induction heads

In *Opening the box* we found the copying circuit by **running** the model: feed it random tokens twice, watch which heads attend where.

- ▶ a **previous-token head** writes “the token before me was X”;
- ▶ an **induction head** finds that mark and copies what follows.

Today we find the same two heads without running **any** text at all - only by multiplying weight matrices. And we get something the running could not give us: a reason *why* it takes two layers.

Optional lecture. It needs only *Opening the box*.

Predict first

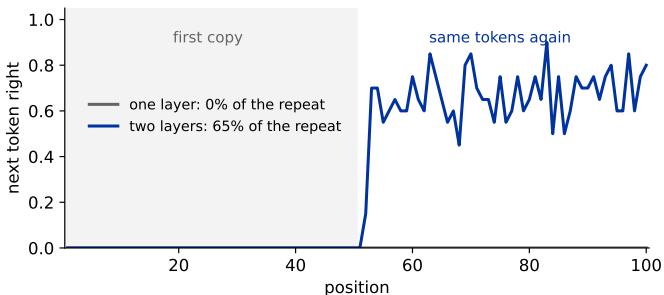
Two tiny transformers, identical except for depth. Give each the same 50 random tokens, twice.

How much of the repeat can a **one-layer** model get right?

And a **two-layer** one?

One layer has eight attention heads. Surely eight heads can look back and copy?

One layer cannot. Two layers can.



20 sequences of 50 random tokens, repeated. **One layer: 0%** of the repeat, and the right token's median rank barely moves (18,122 $\rightarrow$ 15,386 out of 48,262). **Two layers: 65%**, and the median rank of the right token is **1**.

Something about the task needs a head to build on another head's work. By the end of today you can point at the matrices that do it.

The models

attn-only-1l, attn-only-2l

Neel Nanda · TransformerLens

huggingface.co/NeelNanda/Attn_Only_2L512W_C4_Code

layers	1 or 2, no MLPs	width of the stream	512
heads per layer	8	vocabulary	48,262 tokens
trained on	web text (C4) and code		

Why no MLPs: without them, the whole model is **products of matrices**. Products of matrices can be read. An MLP's nonlinearity is exactly what breaks that - which is why this lecture ends where it does.

The method is from Elhage et al. (2021), *A Mathematical Framework for Transformer Circuits* (Anthropic).

Outline

A head is two circuits

Zero layers: a bigram table

One layer: copying

Two layers: composition

Does it survive in GPT-2 small?

What weights can and cannot say

A head is two circuits

Where to look, and what to move - two different matrices

The attention head, written out

From the attention lecture, for a residual-stream row vector x_i at each position i :

$$\begin{aligned} q_i &= x_i W_Q, & k_j &= x_j W_K, & v_j &= x_j W_V \\ \text{score}_{ij} &= q_i \cdot k_j, & a_{ij} &= \text{softmax}_j(\text{score}_{ij}), & \text{out}_i &= \left(\sum_j a_{ij} v_j \right) W_O \end{aligned}$$

Four matrices: W_Q, W_K, W_V are 512×64 , W_O is 64×512 .

The trick of this lecture is to stop thinking of four matrices and regroup them into **two**
- one for each question the head answers.

Where to look: the QK circuit

Write the score out, one step at a time:

$$\begin{aligned}\text{score}_{ij} &= q_i k_j^\top \\ &= (x_i W_Q)(x_j W_K)^\top \\ &= x_i W_Q W_K^\top x_j^\top \\ &= x_i W_{QK} x_j^\top,\end{aligned}$$

dot product as a row times a column

substitute q, k

$$(AB)^\top = B^\top A^\top$$

$$W_{QK} := W_Q W_K^\top \quad (512 \times 512)$$

W_{QK} - the **query-key** (QK) circuit - alone decides **which positions attend to which**.
 W_Q and W_K never appear separately in anything the head does - only their product.

What to move: the OV circuit

Now the output, the same way:

$$\text{out}_i = \left(\sum_j a_{ij} x_j W_V \right) W_O$$

substitute v_j

$$= \sum_j a_{ij} x_j W_V W_O$$

a sum times a matrix

$$= \sum_j a_{ij} x_j W_{OV},$$

$$W_{OV} := W_V W_O \quad (512 \times 512)$$

W_{OV} - the **output-value** (OV) circuit - alone decides **what gets written** once the head has looked. The attention pattern a_{ij} only chooses *how much* of each position to move.

Both matrices are 512×512 but have rank at most 64: a head can only read and write a 64-dimensional slice of the stream.

By hand: a head in a two-number world

Three tokens with 2-number embeddings: $A = (1, 0)$, $B = (0, 1)$, $C = (0.5, -0.5)$.

The current token is A ; the earlier tokens are A , B , C .

$$W_{QK} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \text{ so } A W_{QK} = (0, 1)$$

scores: A : 0, B : 1, C : -0.5

softmax: 0.23, **0.63**, 0.14

A mostly looks at B .

$W_{OV} = I$ (copy what you see):

$$\text{out} = 0.23A + 0.63B + 0.14C = (0.30, 0.56)$$

read out: A : 0.30, **B : 0.56**, C : -0.13

$$W_{OV} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \text{ (swap):}$$

out = $(0.56, 0.30)$ $\rightarrow$ now it predicts **A** .

Same W_{QK} , same attention, two different predictions. **Where it looks** and **what it writes** really are separate - the warning from *Opening the box*, now as two matrices.

Reading the circuits in token space

A 512-dimensional matrix is still unreadable. Sandwich it between the embedding W_E (token $\rightarrow$ stream) and the unembedding W_U (stream $\rightarrow$ logits):

	matrix	entry (s, t) means
QK in token space	$W_E W_{QK} W_E^T$	how much token s wants to attend to token t
OV in token space	$W_E W_{OV} W_U$	what attending to token s does to the logit of t

Both are $48,262 \times 48,262$ - over two billion numbers. Nobody looks at them. But we can **ask them questions**: is the diagonal large? which rows stand out? That is the rest of the lecture.

Zero layers: a bigram table

What can a model predict with no attention at all?

The direct path

Remove every head. The stream at the last position is just the current token's embedding, and the prediction is

$$\text{logits} = \text{LayerNorm}(W_E[\text{token}]) W_U.$$

The next token depends on the current token and **nothing else**. That is a **bigram table**: “after this word, that word” - the simplest language model there is.

Every transformer contains this path, because the residual stream carries the embedding all the way to the end. In the one-layer model it is one term of the sum, and we can read it straight off W_E and W_U .

Reading the bigram table

The one-layer model's direct path, top guesses for a few tokens:

current token	top next token	runner-up
"United"	"States" 99.8%	"Kingdom" 0.1%
"Hong"	"Kong" 96%	"s" 2%
"according"	"to" 99.5%	"same" 0.3%
"New"	"York" 71%	"Zealand" 24%
"San"	"Francisco" 57%	"itation" 17%
"Barack"	"and" 43%	"able" 6%
"import"	"ation" 88%	"able" 8%

The last rows are kept on purpose. The direct path is *one term* of the model's sum - the attention layer adds its own contribution on top, and the model as a whole does not have to get every bigram right through this path alone.

One layer: copying

What eight heads can do, and the one thing they cannot

What one layer adds

With one attention layer the logits are the direct path plus one term per head:

$$\text{logits} = \underbrace{W_E W_U}_{\text{bigram}} + \sum_{\text{heads}} \underbrace{\sum_j a_{ij}}_{\text{QK: where}} \underbrace{W_E W_{OV} W_U}_{\text{OV: what}} [\text{token at } j]$$

So a head at the current position looks back at some earlier token s and adds row s of its OV table to the prediction. Three tokens are involved: the earlier one, the current one, and the predicted one - the paper calls these **skip-trigrams**: “ $s \dots \text{current} \rightarrow \text{next}$ ”.

The most useful skip-trigram of all is also the simplest: *whatever earlier word I look at, predict it again*. Text repeats itself - names, variables, topics. A head that does this is a **copying head**.

How to check a two-billion-entry table

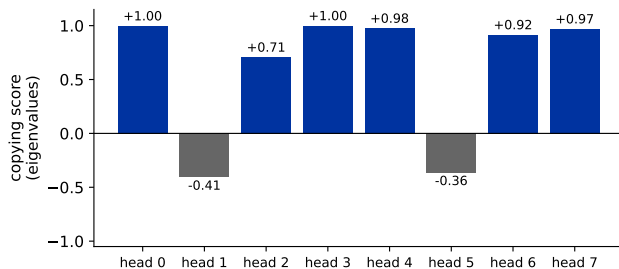
A copying head's OV table $W_E W_{OV} W_U$ has a big diagonal: token t maps to token t . Two ways to test it without printing it:

1. **Directly.** Take 2000 random tokens, compute each one's row, and ask whether the token itself is the top entry.
2. **With eigenvalues.** A matrix that maps things to themselves has mostly *positive* eigenvalues. And we never need the big matrix:

$$\text{the nonzero eigenvalues of } \underbrace{W_E W_{OV} W_U}_{48,262 \times 48,262} = \text{those of } \underbrace{W_{OV} W_U W_E}_{512 \times 512}$$

because ABC and BCA always share their nonzero eigenvalues. Score = $\sum \lambda / \sum |\lambda|$:
+1 means all positive, -1 all negative.

The one-layer model's eight heads



head	0	1	2	3	4	5	6	7
token maps to itself (top-1, directly)	48%	0%	0%	42%	22%	0%	5%	1%

The two tests disagree on heads 2, 6 and 7: eigenvalues near +1, yet almost never mapping a token to itself. A one-number summary of a matrix can look like copying without being copying. Check the direct way too.

Why one layer cannot do induction

To continue "... A B ... A" with B, the head at the second A must attend to the position **right after** the first A.

In one layer, a key at position j is built from the token at j (and its position) - nothing else. Nothing at the position of B says "the token before me was A".

So the head cannot find B by looking for A. The best it can do is copy *every* earlier token a little - which is what the measurement showed: the right token's median rank improves from 18,122 to 15,386, and it is top-1 exactly **0%** of the time.

Some **earlier** head has to write "the token before me was A" into position B's stream first. That needs a second layer.

Two layers: composition

A head in layer 1 can read what a head in layer 0 wrote

Three ways to compose

A layer-1 head reads the stream, and the stream now contains layer-0 heads' outputs.
Which of its inputs does it build from them?

	the layer-1 head's ... uses	matrix to check
	layer-0 output	
Q-composition	query - <i>what it is looking for</i>	$W_{OV}^{(0)} W_{QK}^{(1)}$
K-composition	key - <i>what it finds</i>	$W_{QK}^{(1)} W_{OV}^{(0)\top}$
V-composition	value - <i>what it moves</i>	$W_{OV}^{(0)} W_{OV}^{(1)}$

Induction needs **K-composition**: the induction head's *keys* must carry “the token before me was A”, which only a layer-0 head can have written.

A composition score

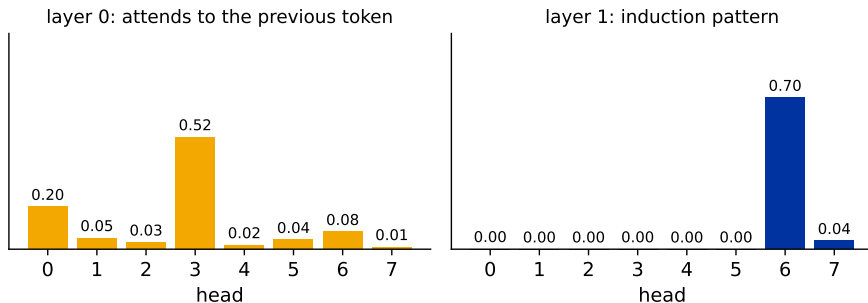
How much does head h_1 read from what head h_0 writes? For K-composition:

$$\text{score}(h_0 \rightarrow h_1) = \frac{\|W_{QK}^{(h_1)} W_{OV}^{(h_0)\top}\|_F}{\|W_{QK}^{(h_1)}\|_F \|W_{OV}^{(h_0)}\|_F}$$

- ▶ $\|\cdot\|_F$ is the Frobenius norm - the square root of the sum of all squared entries.
- ▶ The ratio is between 0 and 1: near 0 when h_0 writes into directions h_1 ignores, larger when they overlap.
- ▶ Random matrices of the same shape score about **0.044** - the baseline to beat.

It is a **norm**, computed from weights. A large score says the two heads *could* talk; it does not say they do, on any particular input.

First, which heads are which



From behaviour, on the repeated random tokens: **L0H3** puts 52% of its attention on the previous token; **L1H6** puts 70% on the position after the previous occurrence of the current token. A previous-token head and an induction head, found by running the model.

Now forget the running. Can the weights alone point at the same pair?

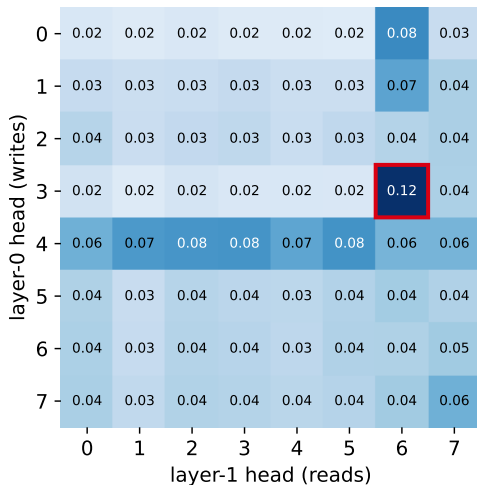
Predict first

There are $8 \times 8 = 64$ possible (layer-0 head $\rightarrow$ layer-1 head) pairs.

If we compute the K-composition score for all 64 from the weights,
where does L0H3 $\rightarrow$ L1H6 rank?

Top ten? Top half? Lost in the noise?

The weights point at the same pair



L0H3 → L1H6 scores **0.124** - **first of all 64**, boxed in red. The median pair scores 0.037; random matrices, 0.044.

The other strong cells in that column are heads 0 and 1 of layer 0 - and L0H0 is the second most previous-token-like head (20%).

Row 4 is mildly high everywhere: that head writes into many directions, so it overlaps a little with every reader.

No text was run to make this picture.
Only W_Q , W_K , W_V , W_O .

Reading the induction head off the weights

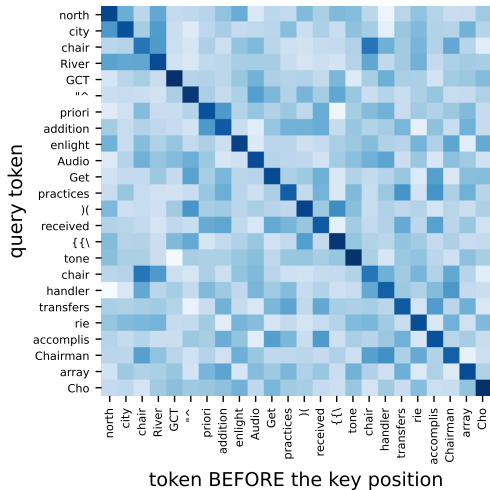
Follow one key through both layers:

1. At position j , the previous-token head L0H3 attends to position $j - 1$ and writes that token through its OV circuit: key input = $W_E[\text{token}_{j-1}] W_{OV}^{L0H3}$.
2. At the current position, the induction head's query is built from the current token: $W_E[\text{token}_i]$.
3. Its score for position j is therefore

$$W_E[\text{token}_i] W_{QK}^{L1H6} (W_E[\text{token}_{j-1}] W_{OV}^{L0H3})^\top$$

A token $\times$ token table: rows are the current token, columns are the token *before* a key. If this is an induction head, the table should be largest where the two are the **same** token - on the diagonal.

The diagonal



A 24×24 corner of that table, for random tokens. The bright diagonal is “attend to where my token came just before”.

Over **300** random tokens, the diagonal is the largest entry in its row for **97%** of them. Chance: 0.3%.

Route the same calculation through any *other* layer-0 head instead of L0H3: 0 to 14%.

The induction head's *search rule*, derived from four weight matrices.

And what it does once it has found the spot

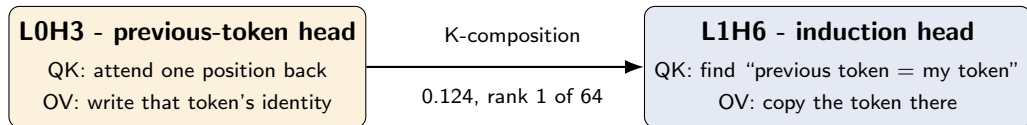
The induction head's OV circuit, tested the direct way on 2000 random tokens:

token maps to itself, top-1	98%
eigenvalue copying score	+1.00

Put the two halves together. **QK**: find the position whose previous token is my token.
OV: copy the token there. That is the whole induction algorithm from *Opening the box*
- found without running a single sentence.

The previous-token head's own OV circuit is *not* copying (score -0.41 , 0% top-1): its output is not meant for the logits at all. It is written for the induction head to read - which is what composition means.

The circuit, assembled

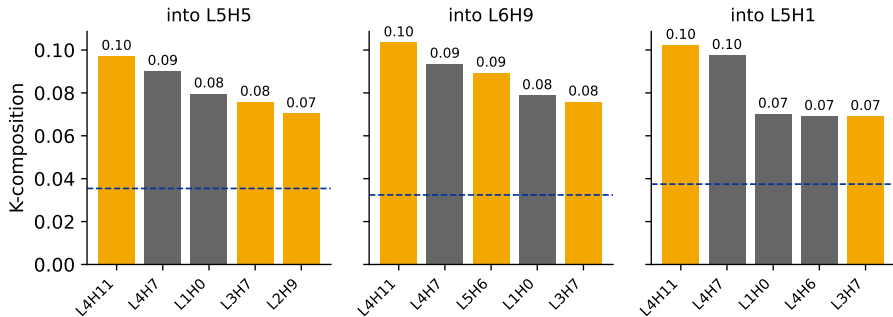


This is why induction needs two layers: the second head's *key* is built out of the first head's *output*. One layer has nothing to build it from.

Does it survive in GPT-2 small?

Twelve layers, MLPs in between, 144 heads

The same question on a real model



For GPT-2's three induction heads (from *Opening the box*): the five earlier heads with the highest K-composition. Orange = attends to the previous token more than 30% of the time. Dashed = the median over all earlier heads.

L4H11 tops all three lists - and it is GPT-2's strongest previous-token head, with **99%** of its attention one position back.

Right answer, much less margin

	toy (2 layers)	GPT-2 small
top source is the previous-token head?	yes	yes, for all three
its score vs the median source	3.4x	2.7-3.2x
runner-up is a previous-token head?	partly (L0H0)	no - L4H7, 18%

L4H7 is almost tied with L4H11 and barely looks at the previous token. From the weights alone you could not tell which of the two is the real partner.

In a real model the weights **suggest**. MLPs sit between the heads, LayerNorm rescales the stream, and 144 heads give thousands of pairs. Deciding between L4H11 and L4H7 takes an intervention - which is what the next core lecture does.

What weights can and cannot say

Exact and data-free - up to the first MLP

The strengths

- ▶ **No data needed.** The composition score and the diagonal test ran on no text at all. What you find holds for *every* input, not the ones you happened to try.
- ▶ **Exact, in attention-only models.** The logits really are sums of products of these matrices - nothing is approximated except LayerNorm's scale.
- ▶ **It explains, not only locates.** Running the model told us *that* two layers are needed. The weights told us *why*: a key built from another head's output.

This is how induction heads were first understood - on models like the two we used today
- before anyone went looking for them in large models.

The limits

- ▶ **MLPs break it.** A nonlinearity between two heads means their combined effect is no longer a product of matrices. Real models have an MLP in every block.
- ▶ **Paths multiply.** Two heads give pairs; three give triples. In GPT-2 small there are 144 heads and every later head can compose with every earlier one.
- ▶ **Scores are norms, not effects.** A large composition score says two heads *could* talk - L4H7 is the proof that it does not mean they do.
- ▶ **Superposition.** A token's embedding is not a clean input when the stream packs many features into each direction - the subject of the features lecture.

So for real models the field moved from reading weights to **intervening on activations**. Weights give you candidates; interventions decide.

Recap

- ▶ A head is two independent circuits: **QK** decides where to look, **OV** decides what to write.
- ▶ No layers: a **bigram table**, read from $W_E W_U$.
- ▶ One layer: **copying** heads - checked directly and with eigenvalues, which disagree on three heads.
- ▶ Two layers: **composition**. The previous-token $\rightarrow$ induction pair is the top K-composition of 64; the induction head's search rule and copying are both readable from the weights.
- ▶ GPT-2 small: the weights point at the right partner, by a hair.

Reading weights explains *why* a circuit works, for every input at once - in models small and linear enough to allow it.

Next core lecture: does it actually do that?

We stop reading and start **intervening**: delete a head, paste an activation from one run into another, and measure what changes. It is the method that works on real models - and it is how you would settle L4H11 against L4H7.